

Список теоретических вопросов и задач к защите лабораторной работы:

1. Роль Clang в экосистеме LLVM

Clang — это фронтенд LLVM для языков C, C++ и Objective-C. Его роль:

- Лексический и синтаксический анализ исходного кода.
- Построение AST (Abstract Syntax Tree).
- Преобразование AST в промежуточное представление **LLVM IR**.
- Выдача диагностических сообщений (ошибки, предупреждения).

2. Форматы вывода Clang

Clang может генерировать:

- **LLVM IR** (текстовый **.ll** или бинарный **.bc**)
- **Ассемблер** (**.s**)
- **Объектные файлы** (**.o**, **.obj**)
- **Исполняемые файлы** (через компоновщик)
- **AST-дамп** (для отладки, **-ast-dump**)

3. Трехуровневая архитектура LLVM

1. **Фронтенд** (Clang, Rustc, Flang) → LLVM IR
2. **Оптимизатор** (Middle-end, **opt**) → улучшенный LLVM IR (в SSA-форме)
3. **Бэкенд** (**llc**, кодогенерация) → машинный код для целевой архитектуры (x86, ARM, RISC-V и др.)

4. Отличие LLVM от традиционных компиляторов

Традиционный компилятор (GCC)

LLVM

Монолитная архитектура	Модульная (фронтенд+opt+бэкенд)
Плохая переиспользуемость	IR может использоваться многими фронтендами
Фронтенд и бэкенд жёстко связаны	Чёткое разделение этапов

5. Отличие в компиляции C и C++ кода в LLVM

- C++ требует более сложной семантики: исключения, RTTI, name mangling, шаблоны.
- Clang генерирует **вызовы библиотечных функций** для поддержки исключений и динамического приведения типов (разные рантаймы: `libc++` или `libstdc++`).
- Уровень AST для C++ богаче (узлы для шаблонов, перегрузок, вложенных областей видимости).

6. Разделение ответственности между Clang и LLVM

Clang (фронтенд)	LLVM (core, оптимизатор, бэкенд)
Парсинг, семантический анализ	Оптимизации (уровни -O1, -O2, -O3)
Построение AST	Преобразование IR
Генерация IR	Генерация машинного кода
Диагностика	Регистровая аллокация, планирование инструкций

7. Отличия Clang и GCC

Clang	GCC
Модульная архитектура (LLVM)	Монолитная
Более понятные сообщения об ошибках	Иногда загадочные
Быстрая компиляция	Медленнее при отладке
Меньшая поддержка старых платформ	Более широкая кросс-платформенность
Лицензия Apache 2.0	GPLv3

8. AST (абстрактное синтаксическое дерево)

AST — это **абстрактное** представление программы, опускающее синтаксические детали (например, точки с запятой, скобки в выражениях, пробелы). Узлы AST — операторы, переменные, константы.

9. Отличие AST от CST

AST	CST (Parse Tree)
Не содержит нетерминалов для пунктуации	Содержит все правила грамматики
Компактнее	Дублирует синтаксис

Используется для семантического анализа

Используется только для синтаксического разбора

10. LLVM IR (промежуточное представление кода). Три формы LLVM IR

1. **Текстовый IR** (читаемый человеком, `.ll`)
2. **Битовый код** (Bitcode, бинарный, `.bc`)
3. **В памяти** (во время работы компилятора)

11. Статическая однократная форма присваивания (SSA)

Каждая переменная **присваивается ровно один раз**. При необходимости слияния путей используются **φ-функции** (phi nodes). Упрощает многие оптимизации (распространение констант, устранение мёртвого кода).

12. CFG (граф потока управления)

Ориентированный граф, где вершины — **базовые блоки** (линейные последовательности инструкций), а рёбра — переходы (условные/безусловные прыжки). Используется для анализа потока управления и оптимизаций.

13. Генерация кода

Этапы бэкенда LLVM:

- **SelectionDAG** → преобразование IR в граф ассемблероподобных операций.
- **Распределение регистров.**
- **Планирование инструкций.**

- **Формирование машинного кода** (MC layer, ассемблер + линковка).

14. Оптимизация кода

Преобразование программы, сохраняющее её семантику, но улучшающее скорость/размер/энергопотребление. LLVM выполняет их как на IR, так и на уровне машинного кода.

15. Уровни оптимизации Clang/LLVM

Уровень	Характеристика
-O0	Нет оптимизаций, быстрая компиляция, отладка
-O1	Базовые оптимизации без роста времени компиляции
-O2	Почти все оптимизации, не увеличивающие сильно размер кода
-O3	Агрессивные оптимизации (векторизация, инлайнинг, разворачивание циклов)
-Os	Оптимизация на размер
-Oz	Ещё сильнее на размер
-Ofast	-O3 с нарушениями строгих стандартов

16. Локальные оптимизации. Примеры

Примеры:

- **Свёртка констант** ($3 + 5 \rightarrow 8$)
- **Устранение мёртвого кода** (удаление недостижимых инструкций)
- **Копирование/распространение констант** ($a = 5; b = a \rightarrow b = 5$)

17. Глобальные оптимизации. Примеры

Примеры:

- **Удаление недостижимых блоков.**
- **Подстановка вызовов простых функций** (inlining — уже межпроцедурная? частично).
- **Анализ активных переменных** для удаления лишних загрузок/сохранений.
- **Loop invariant code motion** (вынос выражений из циклов).

18. Межпроцедурные оптимизации. Примеры

Примеры:

- **Встраивание функций** (inlining, даже косвенное с помощью Devirtualization).
- **Устранение мёртвых функций** (если не вызываются).
- **Постоянное распространение через границы функций** (interprocedural constant propagation).
- **Частичная специализация функций.**
- **Оптимизация во время линковки (LTO).**

Эти оптимизации требуют анализа вызывающих связей и работают над несколькими функциями или даже всем модулем (единицей компиляции).